

Airflow at the center of your pipelines

Learning to orchestrate shouldn't require 3 certifications

The context

When someone wants to start with orchestration, the "traditional" path is: read a thousand docs, spin up an environment, understand a panel, dream of Airflow... and get frustrated. The barrier isn't the concept — it's the tool. With wpipe onboarding is part of the library: a Learning Tour with 140 levels that takes you from the very basics to the most advanced flows.

Why engineers shift to wpipe

- Pipelines are Python: versionable, testable, reviewable in Git.
- State persisted to SQLite WAL at every step; resume, don't restart.
- <50MB RAM. No broker, no daemon, no metadata server.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="task", retry_count=3, retry_delay=2)
def task(data):
    return {"ok": run(data["job"])}

pipe = Pipeline(pipeline_name="airflow_like", tracking_db="jobs.db")
pipe.set_steps([task])

# No scheduler daemon, no heavy metadata DB.
```

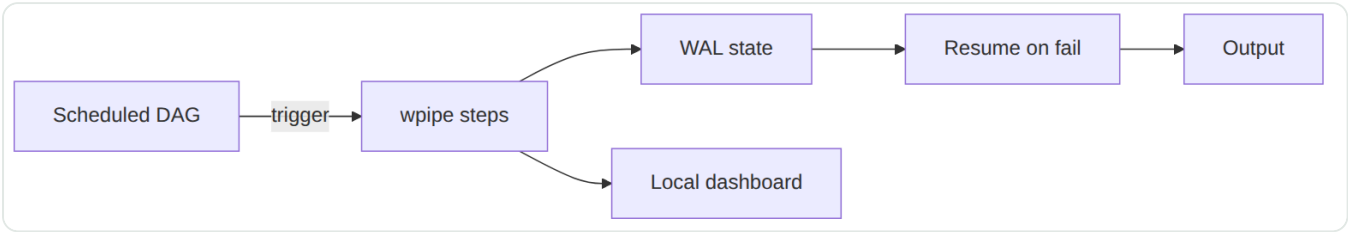
At a glance

Aspect	Legacy / Airflow	wpipe
Definition	Config/DSL or canvas	Python code
State	Scattered / none	SQLite WAL checkpoints
Observability	Partial	Tracker SQL + dashboard

Aspect	Legacy / Airflow	wpipe
Footprint	Heavy	<50MB RAM

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads
A community of efficiency-first developers.

<50MB RAM
Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL
Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **Airflow** workflows at wpipe.readthedocs.io.